



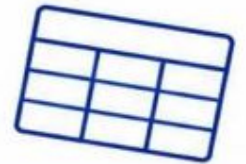
TOP 20

SQL

SCENARIO BASED

INTERVIEW

QUESTIONS

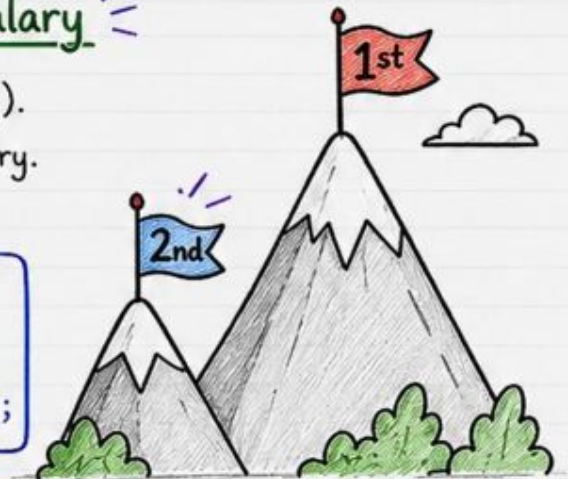


Q1. Find the second highest salary

Scenario: Table employees(emp_id, name, salary).
Return the second highest distinct salary.

Answer:

```
SELECT MAX(salary) AS second_highest_salary
FROM employees
WHERE salary < (SELECT MAX(salary) FROM employees);
```

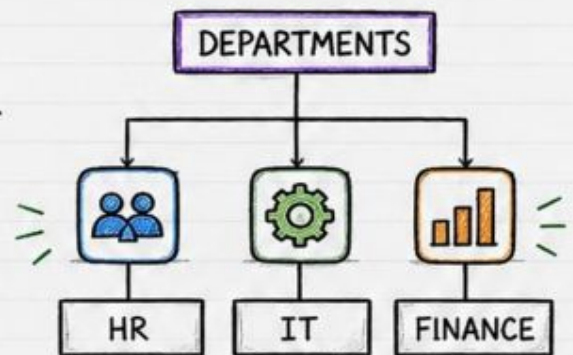


Q2. List all departments

Scenario: Table departments(dept_id, dept_name).
List all departments.

Answer:

```
SELECT dept_id, dept_name
FROM departments;
```



Q3. Count employees in each department

Scenario: employees(emp_id, dept_id).
Return number of employees in each department.

Answer:

```
SELECT dept_id, COUNT(*) AS emp_count
FROM employees
GROUP BY dept_id;
```

DEPT_ID	EMP_COUNT
101	5
102	8
103	3

Q4. Find employees with salary greater than average salary

Scenario: Table employees(emp_id, name, salary).
Find employees earning more than average salary.

Answer:

```
SELECT emp_id, name, salary
FROM employees
WHERE salary > (SELECT AVG(salary) FROM employees);
```



★ Tip: Practice these queries to strengthen your SQL skills!

Q2. Find Employees Who Earn More Than Their Department Average

employees

Scenario: You have an employees table containing:

- emp_id
- emp_name
- dept_id
- salary

Your task is to find employees whose salary is higher than the average salary of their respective department.

emp_id	emp_name	dept_id	salary
101	John	10	60000
102	Alice	10	75000
103	Bob	20	50000
104	Eve	20	80000
105	Mike	30	40000
106	Sara	30	65000
...	...	...	...

SQL Solution:

```
SELECT e.emp_id,
       e.emp_name,
       e.dept_id,
       e.salary
FROM employees e
JOIN (
    SELECT dept_id,
           AVG(salary) AS avg_salary
    FROM employees
    GROUP BY dept_id
) d
ON e.dept_id = d.dept_id
WHERE e.salary > d.avg_salary;
```



Why Interviewers Ask This

- ✓ Aggregate Functions (AVG)
- ✓ GROUP BY Concepts
- ✓ Subqueries & Joins
- ✓ Business Logic Implementation



Real-World Use Case

- Bonus allocation
- Promotion planning
- Compensation benchmarking
- Talent retention programs



Interview Tip

Whenever you see:

"Higher than average", "Lower than average",
"Above department average"

Think of: **Aggregation + Subquery/CTE + Join**

#SQL #DataAnalytics #DataAnalyst #SQLInterviewQuestions #LearningInPublic
#BusinessIntelligence #PowerBI #DataScience #CareerGrowth #CodeBasics

Q3. Find Customers Who Purchased in Every Month of a Quarter

Scenario: You have a sales table:

- customer_id
- order_date
- amount

Find customers who made at least one purchase in every month of a given quarter (e.g., Q1: Jan, Feb, Mar).

SQL Solution:

```
SELECT customer_id
FROM sales
WHERE order_date >= '2024-01-01'
      AND order_date < '2024-04-01'
GROUP BY customer_id
HAVING COUNT(DISTINCT TO_CHAR(order_date, 'Mon')) = 3;
```

Sample Result

customer_id	customer_name
1	Alice
3	Charlie
5	Eve
...	...



Pro Tip

COUNT(DISTINCT TO_CHAR(order_date, 'Mon'))
ensures the customer has at least one order in each month.
(Works for SQL dialects like Oracle/PostgreSQL.)

Q1 (Jan - Mar)		
Jan	Feb	Mar
✓	✓	✓



Explanation

Filter orders in Q1 (Jan to Mar).

- Extract month from order_date.
- Count distinct months per customer.
- Keep customers who have all 3 months.



Variations

- For a year: use 12 months
- For any quarter: adjust date range
- Use **DATE_TRUNC** for more advanced date logic

#SQL #DataAnalytics #DataAnalyst #SQLInterviewQuestions #LearningInPublic
#BusinessIntelligence #PowerBI #DataScience #CareerGrowth #CodeBasics

Q5.

Find Customers Who Haven't Placed Any Orders

Scenario: You have two tables:

- customers(customer_id, customer_name, email)
- orders(order_id, customer_id, order_date, amount)

Find customers who have not placed any orders.

Answer:

```
SELECT c.customer_id,
       c.customer_name,
       c.email
FROM   customers c
LEFT JOIN orders o
      ON c.customer_id = o.customer_id
WHERE  o.order_id IS NULL;
```

customers			orders		
customer_id	customer_name	email	order_id	customer_id	amount
1	Alice	alice@gmail.com	101	1	250
2	Bob	bob@gmail.com	102	3	150
3	Charlie	charlie@gmail.com	103	1	300
4	David	david@gmail.com	104	3	200
5	Eve	eve@gmail.com	...	...	...
...	...	...	...	...	...



Why Interviewers Ask This

- ✓ Tests understanding of LEFT JOIN
- ✓ Identifying missing related records
- ✓ Real-world data quality checks
- ✓ Common in business reporting



Real-World Use Case

- Re-engage inactive customers
- Marketing campaigns
- Improve customer retention



Pro Tip

Use **LEFT JOIN + IS NULL** to find records that don't have a match!

#SQL #DataAnalytics #DataAnalyst #SQLInterviewQuestions #LearningInPublic
#BusinessIntelligence #PowerBI #DataScience #CareerGrowth #CodeBasics

Q6.

Find Duplicate Emails in Users Table

Scenario: You have a users table:

- user_id
- name
- email

Find emails that are registered by more than one user.

Answer:

```
SELECT email,
       COUNT(*) AS occurrence_count
FROM   users
GROUP BY email
HAVING COUNT(*) > 1;
```

users		
user_id	name	email
1	Alice	alice@gmail.com
2	Bob	bob@gmail.com
3	Charlie	alice@gmail.com
4	David	david@gmail.com
5	Eve	bob@gmail.com
6	Frank	frank@gmail.com
...	...	...



Why Interviewers Ask This

- ✓ Tests GROUP BY & HAVING clause
- ✓ Helps in data cleaning & deduplication
- ✓ Important for maintaining data integrity



Real-World Use Case

- Identify duplicate registrations
- Data cleanup
- Prevent communication issues

Sample Result ↘

email	occurrence_count
alice@gmail.com	2
bob@gmail.com	2



Pro Tip

Use **GROUP BY** with **HAVING COUNT(*) > 1** to filter groups with more than one record!



Bonus

To see full user details having duplicate emails, join this result back with users table!

COMING
UP NEXT
➔

Q7. Find Top 3 Products by Revenue in Each Category

Scenario: sales(product_id, product_name, category, amount, sale_date).

Find top 3 products by total sales in each category.



Keep practicing. You've got this!



Q7. Top 3 Products by Revenue in Each Category

Scenario: sales(product_id, product_name, category, amount, sale_date).

Find top 3 products by total sales in each category.

Answer:

```
SELECT product_id, product_name, category, total_sales
FROM (
    SELECT product_id, product_name, category,
           SUM(amount) AS total_sales,
           ROW_NUMBER() OVER(PARTITION BY category
                              ORDER BY SUM(amount) DESC) AS rn
    FROM sales
    GROUP BY product_id, product_name, category
) t
WHERE rn <= 3;
```

sales				
product_id	product_name	category	amount	sale_date
101	Laptop	Electronics	50000	2024-05-01
102	Phone	Electronics	30000	2024-05-02
201	Chair	Furniture	6000	2024-05-01
202	Table	Furniture	8000	2024-05-03
301	T-Shirt	Apparel	500	2024-05-02
...	...	...	...	...



Why Interviewers Ask This

- ✓ Tests Window Functions
- ✓ Ranking & Partitioning
- ✓ Real-world reporting skills
- ✓ Used in leaderboards & dashboards



Real-World Use Case

- Best selling products
- Category performance analysis
- Inventory & demand planning

Q8. Employees with No Projects

Scenario: employees(emp_id, emp_name), projects(project_id, emp_id).

Find employees who are not assigned to any project.

Answer:

```
SELECT e.emp_id, e.emp_name
FROM employees e
LEFT JOIN projects p
    ON e.emp_id = p.emp_id
WHERE p.project_id IS NULL;
```

employees	
emp_id	emp_name
1	John
2	Alice
3	Bob
4	Carol
...	...

projects	
project_id	emp_id
101	1
102	1
103	2
...	...

NO PROJECTS?



Pro Tip

Use **LEFT JOIN + IS NULL** to find unmatched records.

Q9. Customers with More Than 5 Orders in a Month

Scenario: orders(order_id, customer_id, order_date).

Find customers who placed more than 5 orders in any month.

Answer:

```
SELECT customer_id,
       DATE_TRUNC('month', order_date) AS month,
       COUNT(*) AS total_orders
FROM orders
GROUP BY customer_id, DATE_TRUNC('month', order_date)
HAVING COUNT(*) > 5;
```

orders		
order_id	customer_id	order_date
101	1	2024-01-05
102	1	2024-01-18
103	1	2024-01-20
104	1	2024-02-02
105	1	2024-02-15
...	...	...



Real-World Use Case

- Identify active customers
- Campaign targeting
- Customer engagement analysis

Q10. Running Total of Sales

Scenario: sales(sale_id, sale_date, amount).

Find running total of sales by date.

Answer:

```
SELECT sale_date, amount,
       SUM(amount) OVER (ORDER BY sale_date
                          ROWS BETWEEN UNBOUNDED PRECEDING
                          AND CURRENT ROW) AS running_total
FROM sales
ORDER BY sale_date;
```

sales		
sale_id	sale_date	amount
1	2024-05-01	1000
2	2024-05-02	2000
3	2024-05-03	1500
4	2024-05-04	2500
...	...	...



sale_id	sale_date	amount	running_total
1	2024-05-01	1000	1000
2	2024-05-02	2000	3000
3	2024-05-03	1500	4500
4	2024-05-04	2500	7000
...	...	...	...



Why It Matters

- ✓ Used in financial reports
- ✓ Tracks cumulative performance
- ✓ Helps in trend analysis



★ Keep Practicing SQL – Solve, Learn, Repeat! 🚀

Q11. New vs Returning Customers

Scenario: orders(order_id, customer_id, order_date).

Classify customers as 'New' if it's their first order, else 'Returning'.

Answer:

```
SELECT customer_id, order_id, order_date,
       CASE WHEN order_date = first_order THEN 'New'
            ELSE 'Returning' END AS customer_type
FROM (
  SELECT o.*,
         MIN(order_date) OVER(PARTITION BY customer_id) AS first_order
  FROM orders o
) t
ORDER BY customer_id, order_date;
```

orders		
order_id	customer_id	order_date
101	1	2024-01-01
102	1	2024-02-10
201	2	2024-01-05
202	2	2024-03-15
301	3	2024-02-20
...	...	...

customer_id	order_id	order_date	customer_type
1	101	2024-01-01	New
1	102	2024-02-10	Returning
2	201	2024-01-05	New
2	202	2024-03-15	Returning
3	301	2024-02-20	New
...	...	...	...



Why Interviewers Ask This

- ✓ Window functions
- ✓ First/last event detection
- ✓ Customer lifecycle analysis
- ✓ Used for cohorts & retention



Real-World Use Case

- Retention strategies
- Email campaigns
- Customer segmentation
- Lifetime value analysis



Pro Tip

MIN(order_date) OVER(PARTITION BY customer_id) gives the first order date for each customer. Compare each order_date with first_order to classify!



Q12. Nth Highest Salary in the Company

Scenario: employees(emp_id, emp_name, salary).

Find the 3rd highest distinct salary.

Answer:

```
SELECT DISTINCT salary
FROM employees
ORDER BY salary DESC
LIMIT 1 OFFSET 2;
```

employees		
emp_id	emp_name	salary
1	John	90000
2	Alice	70000
3	Bob	70000
4	Eve	60000
5	Mike	50000
6	Sara	40000



Explanation

- DISTINCT → remove duplicates
- ORDER BY DESC → highest first
- OFFSET 2 → skip top 2 salaries
- LIMIT 1 → return the 3rd highest

Result (3rd Highest Salary)

60000



Pro Tip

You can also use DENSE_RANK() if interviewer expects ranking logic:

```
SELECT salary FROM (SELECT salary, DENSE_RANK() OVER(ORDER BY salary DESC) AS rn FROM employees) t
WHERE rn = 3;
```



Q13. Departments with Highest Average Salary

Scenario: employees(emp_id, emp_name, dept_id, salary).

Find the department(s) having the highest average salary.

Answer:

```
SELECT dept_id, AVG(salary) AS avg_salary
FROM employees
GROUP BY dept_id
HAVING AVG(salary) = (
  SELECT MAX(avg_salary)
  FROM (
    SELECT AVG(salary) AS avg_salary
    FROM employees
    GROUP BY dept_id
  ) x
);
```

employees			
emp_id	emp_name	dept_id	salary
1	John	10	90000
2	Alice	10	70000
3	Bob	20	80000
4	Eve	20	85000
5	Mike	30	50000
6	Sara	30	55000

Result

dept_id	avg_salary
20	82500



Why Interviewers Ask This

- ✓ Aggregation & GROUP BY
- ✓ Subqueries
- ✓ Business insight extraction
- ✓ Identify best-performing groups



Real-World Use Case

- Budget allocation
- Performance benchmarking
- Compensation planning
- Department analysis



Keep Practicing SQL - Solve. Learn. Repeat!



Q14.

Consecutive Login Days by User

Scenario: Table: login_logs(user_id, login_date)
Find users who logged in for at least 3 consecutive days.

Answer:

```
SELECT user_id
FROM (
    SELECT user_id,
           login_date,
           login_date - INTERVAL '1 day' *
              ROW_NUMBER() OVER(PARTITION BY user_id
                                ORDER BY login_date) AS grp
    FROM login_logs
) t
GROUP BY user_id, grp
HAVING COUNT(*) >= 3;
```

login_logs	
user_id	login_date
1	2024-05-01
1	2024-05-02
1	2024-05-03
1	2024-05-05
2	2024-05-01
2	2024-05-02
2	2024-05-03
2	2024-05-04
3	2024-05-02
3	2024-05-04
...	...

Result (Users with 3+ Consecutive Days)

user_id
1
2



Why Interviewers Ask This

- ✓ Tests window functions
- ✓ Advanced logic building
- ✓ Real-world user activity analysis
- ✓ Useful for streak / engagement reports



Real-World Use Case

- Track user engagement streaks
- Identify active users
- Gamification & rewards
- SaaS product analytics



Pro Tip

Subtracting ROW_NUMBER() * interval creates groups of consecutive dates with the same value.

Q15.

Products Never Purchased by Any Customer

Scenario: Tables: products(product_id, product_name)
orders(order_id, product_id)
Find products that have never been purchased.

Answer:

```
SELECT p.product_id, p.product_name
FROM products p
LEFT JOIN orders o
  ON p.product_id = o.product_id
WHERE o.product_id IS NULL;
```

products	
product_id	product_name
1	Laptop
2	Mobile
3	Tablet
4	Headphones
5	Smartwatch
...	...

orders	
order_id	product_id
101	1
102	2
103	1
104	3
105	2
...	...

Result (Never Purchased Products)

product_id	product_name
4	Headphones
5	Smartwatch



Why Interviewers Ask This

- ✓ Tests LEFT JOIN + NULL logic
- ✓ Find missing relationships
- ✓ Important for inventory & catalog analysis



Real-World Use Case

- Identify unsold products
- Inventory optimization
- Marketing campaign targeting



Pro Tip

LEFT JOIN + WHERE right_table.column IS NULL is the key to find missing records.

Q16.

Second Highest Salary in Each Department

Scenario: Table: employees(emp_id, emp_name, dept_id, salary)
Find the second highest salary in each department.

Answer:

```
SELECT dept_id, salary AS second_highest_salary
FROM (
    SELECT dept_id, salary,
           DENSE_RANK() OVER(PARTITION BY dept_id
                              ORDER BY salary DESC) AS rnk
    FROM employees
) t
WHERE rnk = 2;
```

employees			
emp_id	emp_name	dept_id	salary
1	John	10	90000
2	Alice	10	70000
3	Bob	10	70000
4	Eve	20	85000
5	Mike	20	60000
6	Sara	30	50000
7	Tom	30	40000
...	...	...	...

Result (Second Highest Salary)	
dept_id	second_highest_salary
10	70000
20	60000
30	40000



Why Interviewers Ask This

- ✓ Tests ranking functions
- ✓ Department-wise analysis
- ✓ Important for compensation insights



Real-World Use Case

- Identify top earners (excluding highest)
- Benchmark salaries
- Performance & reward planning



Pro Tip

Use DENSE_RANK() if there can be tie salaries.
Use ROW_NUMBER() if you want exactly one second highest.



Keep Practicing SQL – Solve, Learn, Repeat!



Q17 Month-over-Month Revenue Growth



Scenario:

Table: sales(sale_id, sale_date, amount)
Calculate month-over-month (MoM) revenue and its growth percentage.



Answer (SQL):

```
SELECT
    month,
    revenue,
    LAG(revenue) OVER (ORDER BY month)
    AS prev_month_revenue,
    ROUND(
        (revenue - LAG(revenue) OVER (ORDER BY month)) * 100.0
        / NULLIF(LAG(revenue) OVER (ORDER BY month), 0), 2
    ) AS growth_percent
FROM (
    SELECT DATE_TRUNC('month', sale_date) AS month,
           SUM(amount) AS revenue
    FROM sales
    GROUP BY DATE_TRUNC('month', sale_date)
) t
ORDER BY month;
```

sales

sale_id	sale_date	amount
1	2024-01-15	1000
2	2024-01-20	1500
3	2024-02-05	2000
4	2024-02-18	3000
5	2024-03-10	2500
6	2024-03-25	3500
...	...	...



Result

month	revenue	prev_month_revenue	growth_percent
2024-01-01	2500	NULL	NULL
2024-02-01	5000	2500	100.00
2024-03-01	6000	5000	20.00
...	...	...	...



Why Interviewers Ask This

- ✓ Tests window functions (LAG)
- ✓ Financial reporting skills
- ✓ Trend & growth analysis
- ✓ Real-time business insights



Pro Tip

Use **LAG()** to compare current row with the previous row.



Q18 Active Users in the Last 7 Days



Scenario:

Table: user_activity(user_id, activity_date)
Find users who were active at least once in each of the last 7 days (including today).



Answer (SQL):

```
SELECT user_id
FROM user_activity
WHERE activity_date >= CURRENT_DATE - INTERVAL '6 day'
GROUP BY user_id
HAVING COUNT(DISTINCT activity_date) = 7;
```

user_activity

user_id	activity_date
1	2024-05-20
1	2024-05-21
1	2024-05-22
1	2024-05-23
1	2024-05-24
1	2024-05-25
1	2024-05-26
2	2024-05-20
2	2024-05-21
2	2024-05-23
...	...

Result

user_id
1



Why Interviewers Ask This

- ✓ Tests GROUP BY + HAVING
- ✓ Distinct count logic
- ✓ User engagement analysis
- ✓ Common in SaaS / Product analytics



Real-World Use Case

- Identify highly engaged users
- Retention analysis
- Feature adoption tracking
- Active user dashboards



Pro Tip

COUNT(DISTINCT activity_date) = 7 ensures the user was active on all 7 unique days.



Keep Practicing SQL – Solve, Learn, Repeat!



Q19

Customers Who Spent More Than Average in Their Country



Scenario:

Tables:

- customers(customer_id, customer_name, country)
- orders(order_id, customer_id, amount)

Find customers who spent more than the average spend of customers in their country.



Answer (SQL):

```
SELECT c.customer_id, c.customer_name, c.country,
       SUM(o.amount) AS total_spent
FROM   customers c
JOIN   orders o ON c.customer_id = o.customer_id
GROUP BY c.customer_id, c.customer_name, c.country
HAVING SUM(o.amount) > (
    SELECT AVG(total_amount)
    FROM (
        SELECT c2.country, o2.customer_id,
               SUM(o2.amount) AS total_amount
        FROM   customers c2
        JOIN   orders o2 ON c2.customer_id = o2.customer_id
        GROUP BY c2.country, o2.customer_id
    ) t
    WHERE t.country = c.country
);
```

customers			orders		
customer_id	customer_name	country	order_id	customer_id	amount
1	Alice	USA	101	1	120
2	Bob	USA	102	1	80
3	Charlie	UK	103	2	50
4	David	UK	104	3	200
5	Eva	India	105	4	80
...	...	...	...	...	...



Result			
customer_id	customer_name	country	total_spent
1	Alice	USA	200
3	Charlie	UK	200
5	Eva	India	300
...	...	...	...



Why Interviewers Ask This

- ✓ Tests subqueries + aggregation
- ✓ Multi-table analysis
- ✓ Business logic implementation
- ✓ Benchmarking & comparison skills
- ✓ Customer segmentation



Real-World Use Case

- High value customer identification
- Country-wise performance analysis
- Sales strategy & personalization
- Customer segmentation
- Marketing campaign targeting



Pro Tip

We compare each customer's total spend with the average spend of customers in the same country using a correlated subquery.



Q20

Employees with Salary Higher Than Department Average



Scenario:

Table:

employees(emp_id, emp_name, dept_id, salary)
Find employees whose salary is higher than the average salary of their department.



Answer (SQL):

```
SELECT emp_id, emp_name, dept_id, salary
FROM   employees e
WHERE  salary > (
    SELECT AVG(salary)
    FROM   employees
    WHERE  dept_id = e.dept_id
);
```

employees			
emp_id	emp_name	dept_id	salary
1	John	10	90000
2	Alice	10	70000
3	Bob	10	70000
4	Eve	20	85000
5	Mike	20	60000
6	Sara	20	50000
7	Tom	30	40000
...	...	...	...

Result			
emp_id	emp_name	dept_id	salary
1	John	10	90000
4	Eve	20	85000
5	Mike	20	60000
...	...	...	...



Why Interviewers Ask This

- ✓ Tests correlated subqueries
- ✓ Department-wise analysis
- ✓ Salary benchmarking
- ✓ Business logic implementation



Real-World Use Case

- Performance evaluation
- Employee compensation review
- Identifying top performers
- Budget planning & increments



Pro Tip

The subquery gets the average salary for the same department and compares it with each employee.



Keep Practicing SQL – Solve, Learn, Repeat!

